

RCAi

Root Cause Analysis Intelligence — Technical Documentation

This document explains how the RCAi codebase is structured, how data moves through the system end to end, and how each file connects to the others. It is meant to be read alongside the code in VS Code, and separately from the per-incident RCA report PDF that the running application itself generates.

1. CONNECTORS	2. NORMALIZE	3. CORRELATE	4. DETECT	5. Ai REASONING	6. REPORT
CloudWatch Site24x7 Log / CSV / JSON upload Manual notes	Event objects (timestamp, source, severity, message)	Group events into time-proximity clusters	Statistical anomaly detection (spikes, escalations, bursts)	Claude generates RCA JSON from the correlated evidence	Editable in browser → export PDF / DOCX

Figure 1 — the RCAi pipeline, left to right

1. What RCAi does

Engineers currently pull data from CloudWatch, Site24x7, application logs, and alert tools by hand, then manually piece together what happened during an incident. RCAi automates the mechanical parts of that process — collecting, normalizing, and correlating the data — and then hands a clean, evidence-linked timeline to an AI model for the reasoning step. The result is a structured RCA report where every claim is labeled as either directly supported by data, or the model's own inference.

1.1 Why the evidence / inference split matters

The single design decision that shapes the whole codebase is this: the system never lets the AI's reasoning masquerade as fact. Every deterministic step (correlation, anomaly detection) produces evidence that is fed to the model, and the model's prompt requires it to tag its own output as either **confirmed_evidence** (traceable to a specific event or anomaly) or **ai_inference** (its own reasoning). This shows up consistently in the UI, the PDF, and the DOCX exports as color-coded tags.

2. Architecture

RCAi is a single Flask application that serves both the API and the frontend, running entirely on your machine at 127.0.0.1. There is no separate frontend build step — the browser talks directly to Flask via `fetch()` calls to JSON endpoints.

Layer	Location	Responsibility
Frontend	frontend/templates, frontend/static	Renders the 4-step UI, calls the API, lets the user edit the RCA before export
API layer	backend/app.py	Flask routes: create incidents, ingest data, trigger analysis, serve exports
Connectors	backend/connectors/	Turn each external tool's data into normalized Event objects
Correlation engine	backend/engine/correlator.py	Groups events into time-based clusters
Anomaly engine	backend/engine/anomaly_detector.py	Flags statistical outliers with a stated reason
AI engine	backend/engine/ai_analyzer.py	Sends correlated evidence to Claude, parses structured JSON back
Exporters	backend/exporters/	Render the final RCA into PDF and DOCX
Storage	data/reports/, data/uploads/	JSON snapshots of each incident; uploaded files

3. The data model: everything becomes an Event

The reason RCAi can mix CloudWatch alarms, Site24x7 alerts, and raw log lines in one timeline is that every connector converts its native format into the same small structure, defined in `backend/models/incident.py`:

```
Event(timestamp, source, severity, message, resource,
        metric_name, metric_value, raw)
```

An Incident object holds the metadata for one investigation (title, description, time window) plus the list of Events collected for it, and — once generated — the full RCA report. Incidents are persisted as JSON files under `data/reports/<incident_id>.json`, so closing the browser or restarting the server does not lose your work.

4. Connectors: pulling in evidence

Every connector inherits from BaseConnector (backend/connectors/base_connector.py) and implements a single method: `fetch_events(start_time, end_time)`. This is the plug-in point for adding new tools.

4.1 CloudWatch connector

backend/connectors/cloudwatch_connector.py checks whether AWS credentials are present in your .env file. If they are, it uses boto3 to call the real CloudWatch describe_alarms API. If not, it returns a realistic demo timeline (CPU/memory spikes, health-check failures, autoscaling events) so the rest of the pipeline can be tried immediately without any AWS account.

4.2 Site24x7 connector

backend/connectors/site24x7_connector.py follows the same live/demo pattern, calling the Site24x7 REST alerts API when SITE24X7_API_KEY is set, and returning demo uptime/response-time alerts otherwise.

4.3 Log / file upload parser

backend/connectors/log_parser.py handles anything a user drags in: plain-text app logs (one Event per line, with severity guessed from keywords like 'error' or 'timeout', and timestamps extracted with regex), JSON exports (arrays of alert/log objects), and CSV alert dumps. This is the escape hatch for any monitoring tool that doesn't have a dedicated connector yet.

5. Correlation and anomaly detection

These two modules run before the AI ever sees the data, and they are intentionally simple and explainable rather than black-box.

5.1 Correlator (backend/engine/correlator.py)

Sorts all events chronologically, then chains them into clusters: any event within 120 seconds (configurable) of the previous event in a cluster joins that cluster; otherwise a new cluster starts. Each cluster records its time span, which sources contributed to it, its peak severity, and its event count. This is what lets the app say 'these 13 events across CloudWatch, Site24x7 and your app log all happened within the same two-minute window' — the first real signal that they're related.

5.2 Anomaly detector (backend/engine/anomaly_detector.py)

- **Metric spikes** — for any metric with multiple numeric readings, flags points more than 1.5 standard deviations from the mean (a z-score check).
- **Severity escalations** — flags any point where severity jumps two or more levels between consecutive events (e.g. warning straight to critical).
- **Event bursts** — flags clusters with an unusually high event count compared to the rest of the incident window.

Every anomaly returned includes an `explanation` field in plain English, because the point of RCAi is explainability, not a black-box score.

6. The AI reasoning layer

`backend/engine/ai_analyzer.py` is the only place the codebase talks to an LLM. It never lets the model free-associate about the incident — it always passes the exact correlated clusters and detected anomalies from the previous two steps, and a strict system prompt that requires a fixed JSON schema in return: `executive_summary`, `confirmed_evidence[]`, `root_cause`, `contributing_factors[]`, `incident_impact`, `resolution_actions[]`, `preventive_actions[]`, and `timeline_narrative`.

Every `confirmed_evidence` item must cite the specific cluster or anomaly it came from (e.g. 'Cluster #2 / CloudWatch CPUUtilization event'), and the root cause is always explicitly labeled as `ai_inference`. If no `ANTHROPIC_API_KEY` is configured, or the API call fails for any reason, `generate_rca_fallback()` produces a rule-based report from the same clusters/anomalies so the app degrades gracefully instead of breaking.

7. API reference

Method & path	Purpose
POST /api/incidents	Create a new incident investigation
GET /api/incidents	List all saved incidents (for the sidebar)
GET /api/incidents/<id>	Get full state of one incident
POST /api/incidents/<id>/fetch/cloudwatch	Pull CloudWatch events for the incident window
POST /api/incidents/<id>/fetch/site24x7	Pull Site24x7 events for the incident window
POST /api/incidents/<id>/upload	Upload and parse a log / CSV / JSON file

POST /api/incidents/<id>/manual-event	Add a manually typed observation
POST /api/incidents/<id>/generate-rca	Run correlation + anomaly detection + AI RCA
PUT /api/incidents/<id>/rca	Save the user's edits to the RCA before export
GET /api/incidents/<id>/export/<fmt>	Download the report as pdf, docx, or json
GET /api/health	Check whether AI / CloudWatch / Site24x7 are live-configured

8. Exporting: PDF and editable DOCX

backend/exporters/pdf_exporter.py builds the final, shareable PDF using reportlab's Platypus layout engine — the same report structure the browser shows, rendered as a formal document with an evidence table and an appendix of raw clusters/anomalies. backend/exporters/docx_exporter.py builds the same content as a Word document using python-docx, so the report can be edited directly in Word or Google Docs after download — useful if a manager or another team needs to add context before it's shared further.

Both exporters read from the same in-memory report structure (incident + clusters + anomalies + rca), so anything the user edits in the browser (via the contenteditable fields) is saved back through PUT /rca before either export is generated — what you download always matches what's on screen.

9. Running it locally

```
cd backend
pip install -r requirements.txt
cp .env.example .env # then fill in your Anthropic API key
python3 app.py
# open http://127.0.0.1:5000
```

9.1 Adding a new monitoring source

- 1 Create backend/connectors/your_tool_connector.py, subclassing BaseConnector.
- 2 Implement fetch_events() to return a list of Event objects.
- 3 Add a Flask route in app.py, mirroring /fetch/cloudwatch.
- 4 Add a source card in index.html and wire its button in app.js.

Nothing else needs to change — correlation, anomaly detection, and AI reasoning all operate on the normalized Event list regardless of where it came from.

9.2 Security notes for production use

This ships configured for local, single-user use (Flask's development server, no authentication). Before exposing it beyond localhost: put it behind a production WSGI server (gunicorn/uwsgi), add authentication, move the JSON file store to a real database, and keep API keys out of version control (the provided .gitignore-style convention is to commit .env.example only, never .env).